



University
of Exeter

COMM113

Deep Learning – Week 5

Convolutional Neural Networks

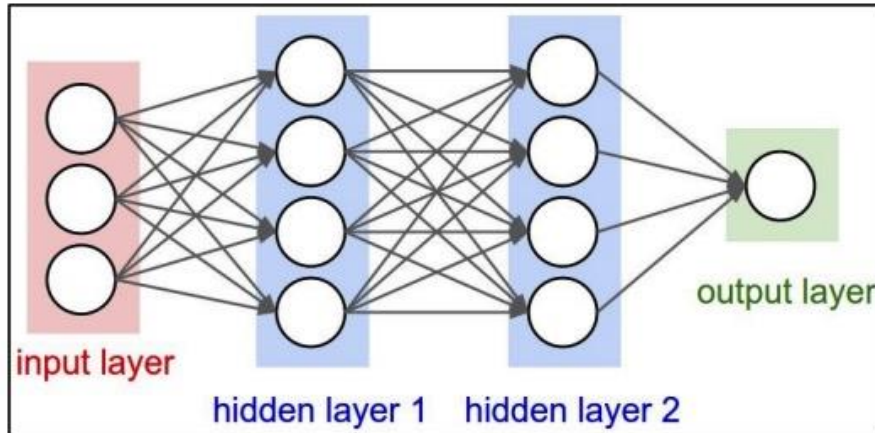
Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Mini-batch Optimisation

Loop:

1. Sample a batch of data
2. Forward prop it through the net, get loss
3. Backprop to calculate the gradients
4. Update the parameters using the gradient

What's the purpose of using *mini-batches* instead of the full dataset?



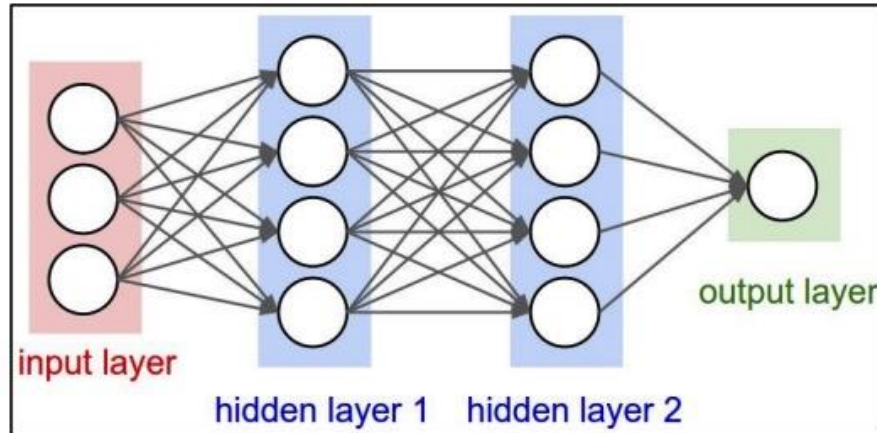
Mini-batch Optimisation

Loop:

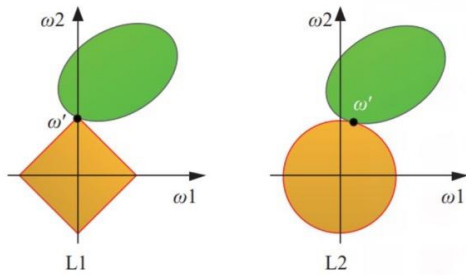
1. Sample a batch of data
2. Forward prop it through the net, get loss
3. Backprop to calculate the gradients
4. Update the parameters using the gradient

What's the purpose of using *mini-batches* instead of the full dataset?

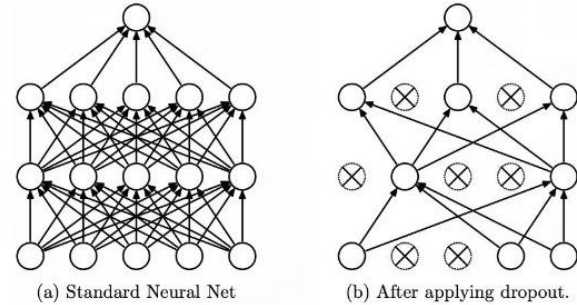
computational efficiency



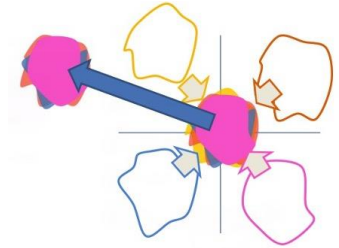
Regularisers



L1/L2



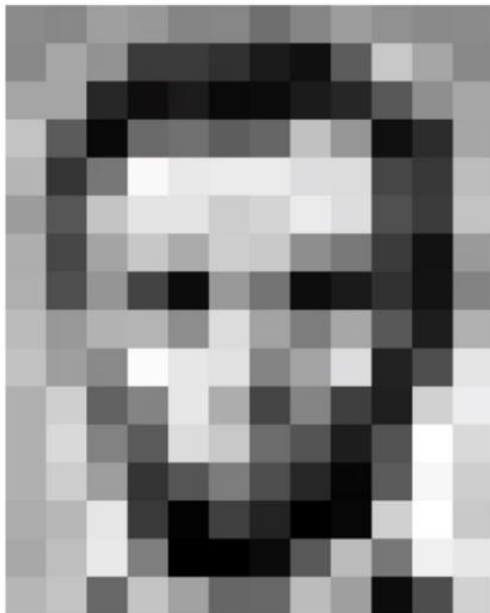
Dropout



Batch Normalisation

What computers 'see': Images as Numbers

What you see



Input Image

What you both see

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	94	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Input Image + values

What the computer "sees"

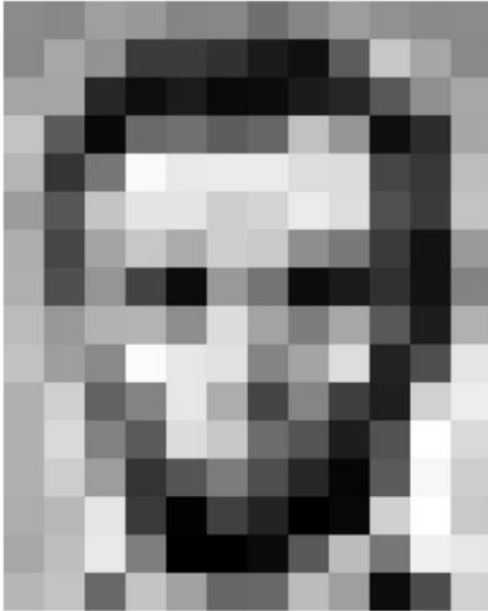
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel intensity values
("pix-el"=picture-element)

This is what computers see — arrays of numbers, not colours or shapes. Each pixel in an image corresponds to a number, often intensity or RGB values.

What computers 'see': Images as Numbers

What you see



Input Image

What you both see

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	94	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Input Image + values

What the computer "sees"

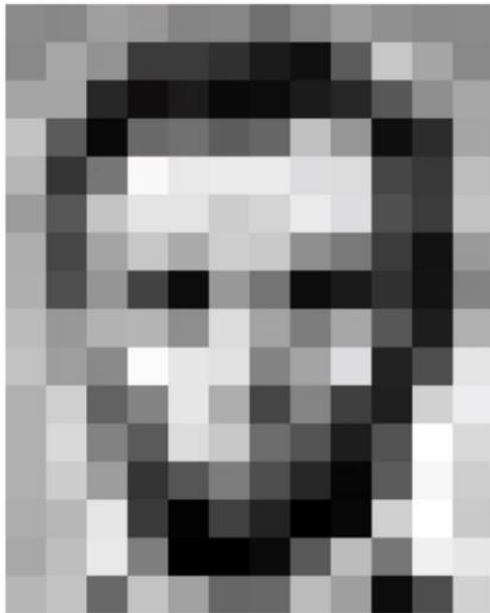
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel intensity values
("pix-el"=picture-element)

If we flatten a $32 \times 32 \times 3$ image into a 1D vector, what information do we lose?

What computers 'see': Images as Numbers

What you see



Input Image

What you both see

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	94	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Input Image + values

What the computer "sees"

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Pixel intensity values
("pix-el"=picture-element)

Levin Image Processing & Computer Vision

If we flatten a $32 \times 32 \times 3$ image into a 1D vector, what information do we lose?

Spatial structure — relationships between neighbouring pixels.

High Level Feature Detection

Let's identify key features in each image category



Nose, Eyes, Mouth



Wheels, License Plate,
Headlights



Door, Windows, Steps

Why might we not want every neuron connected to every pixel?

High Level Feature Detection

Let's identify key features in each image category



Nose, Eyes, Mouth



Wheels, License Plate,
Headlights

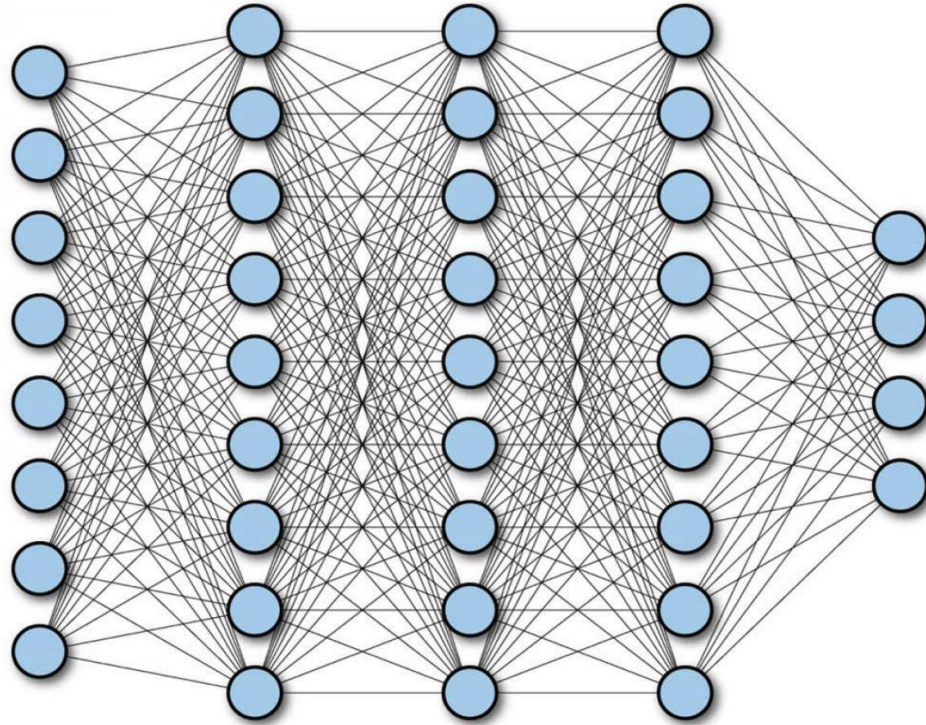


Door, Windows, Steps

Why might we not want every neuron connected to every pixel?

That would ignore spatial information and increase parameters.

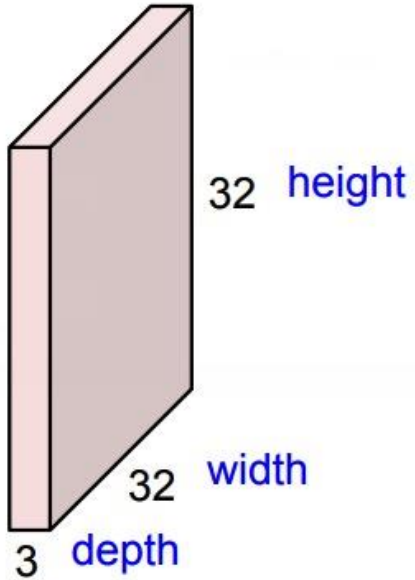
Fully Connected Net



May not get spatial information!

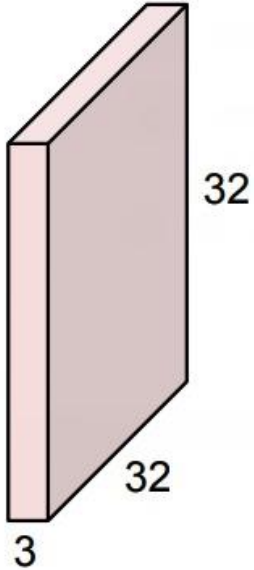
Convolution Layer

32x32x3 image



Convolution Layer

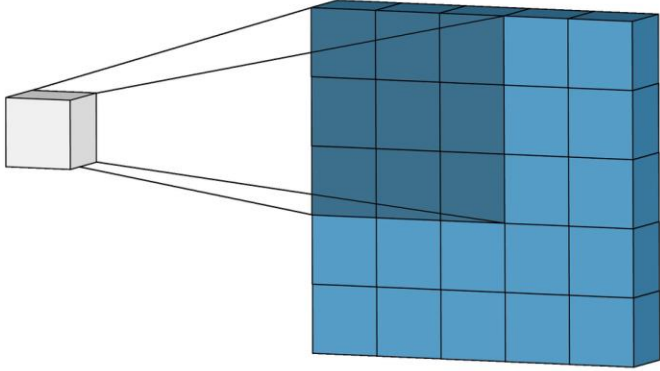
32x32x3 image



5x5x3 filter



Convolution Operation



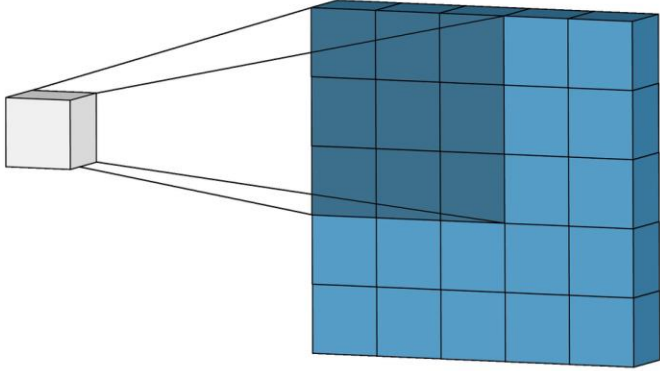
Slide the filter step by step on the input image.

Output: y

Input: x

Filter: w

Convolution Operation



Slide the filter step by step on the input image.

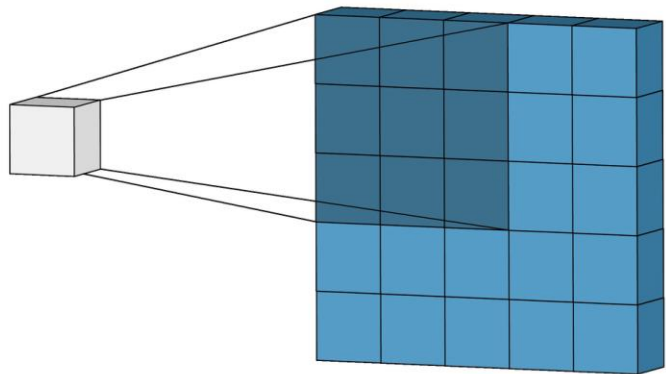
Input 5×5 , Filter $3 \times 3 \rightarrow$

Output: y

Input: x

Filter: w

Convolution Operation



Output: y

Input: x

Filter: w

Slide the filter step by step on the input image.

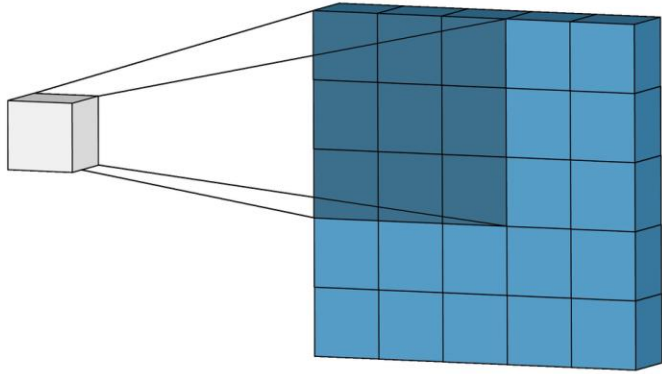
Input 5×5 , Filter $3 \times 3 \rightarrow$

Output 3×3 (no padding, stride = 1)

$$\begin{aligned} y(0,0) = & x(0,0)w(0,0) + x(0,1)w(0,1) + x(0,2)w(0,2) \\ & + x(1,0)w(1,0) + x(1,1)w(1,1) + x(1,2)w(1,2) \\ & + x(2,0)w(2,0) + x(2,1)w(2,1) + x(2,2)w(2,2) \end{aligned}$$

$$\begin{aligned} y(1,1) = & x(1,1)w(0,0) + x(1,2)w(0,1) + x(1,3)w(0,2) \\ & + x(2,1)w(1,0) + x(2,2)w(1,1) + x(2,3)w(1,2) \\ & + x(3,1)w(2,0) + x(3,2)w(2,1) + x(3,3)w(2,2). \end{aligned}$$

Convolution Operation



Output: y

Input: x

Filter: w

After linear operation?

Slide the filter step by step on the input image.

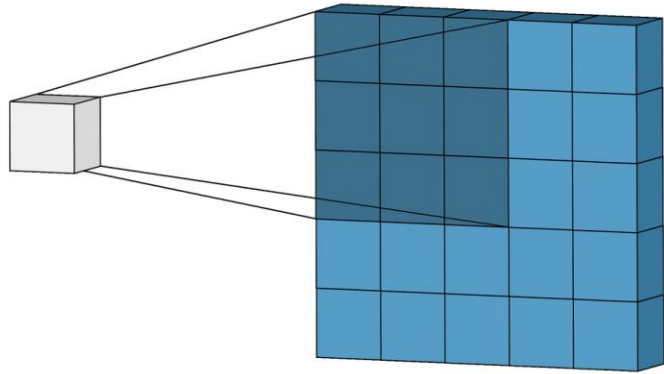
Input 5×5 , Filter $3 \times 3 \rightarrow$

Output 3×3 (no padding, stride = 1)

$$\begin{aligned} y(0,0) = & x(0,0)w(0,0) + x(0,1)w(0,1) + x(0,2)w(0,2) \\ & + x(1,0)w(1,0) + x(1,1)w(1,1) + x(1,2)w(1,2) \\ & + x(2,0)w(2,0) + x(2,1)w(2,1) + x(2,2)w(2,2) \end{aligned}$$

$$y(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} x(i+m, j+n) \cdot w(m,n)$$

Convolution Operation



Slide the filter step by step on the input image.

Input 5×5 , Filter $3 \times 3 \rightarrow$

Output 3×3 (no padding, stride = 1)

$$\begin{aligned} y(0,0) = & x(0,0)w(0,0) + x(0,1)w(0,1) + x(0,2)w(0,2) \\ & + x(1,0)w(1,0) + x(1,1)w(1,1) + x(1,2)w(1,2) \\ & + x(2,0)w(2,0) + x(2,1)w(2,1) + x(2,2)w(2,2) \end{aligned}$$

$$y(i,j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} x(i+m, j+n) \cdot w(m,n)$$

Output: y

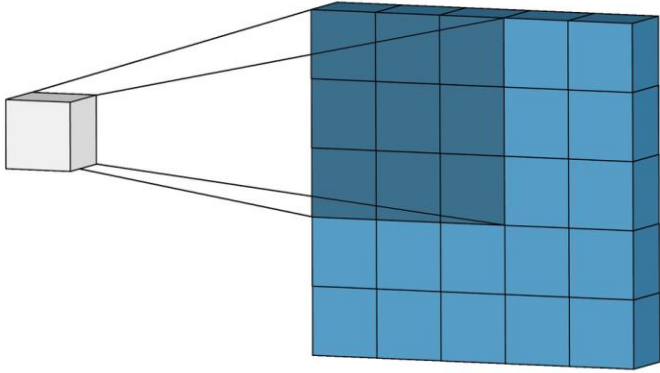
Input: x

Filter: w

After linear operation?

Apply activation function to output y.

Convolution Operation



Stride: how much the filter moves each step.

Input 5×5 , Filter $3 \times 3 \rightarrow$
Output 3×3 (no padding, stride = 1)

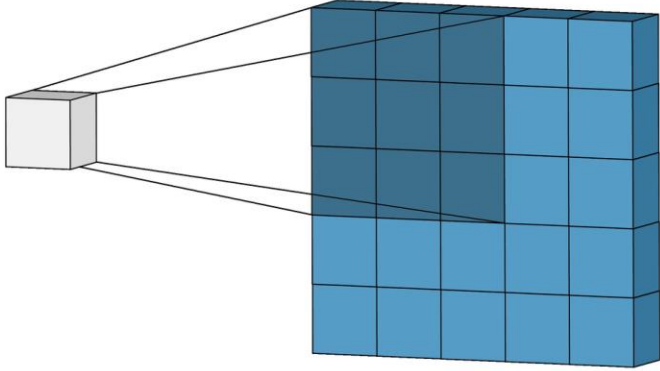
What if stride = 2?

Output: y

Input: x

Filter: w

Convolution Operation



Stride: how much the filter moves each step.

Input 5×5 , Filter $3 \times 3 \rightarrow$
Output 3×3 (no padding, stride = 1)

What if stride = 2?

Input 5×5 , Filter $3 \times 3 \rightarrow$
Output 2×2 (no padding, stride = 2)

Convolution Operation

Padding = 1

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

Zero-Padding: adding zeros to the input

What's the meaning of padding?

Convolution Operation

Padding = 1

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

Padding: adding zeros to the input

What's the meaning of padding?
Can keep the size.

Convolution Operation

Padding = 1

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

Why might we want to keep the output size the same as input?

Padding: adding zeros to the input

Convolution Operation

Padding = 1

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

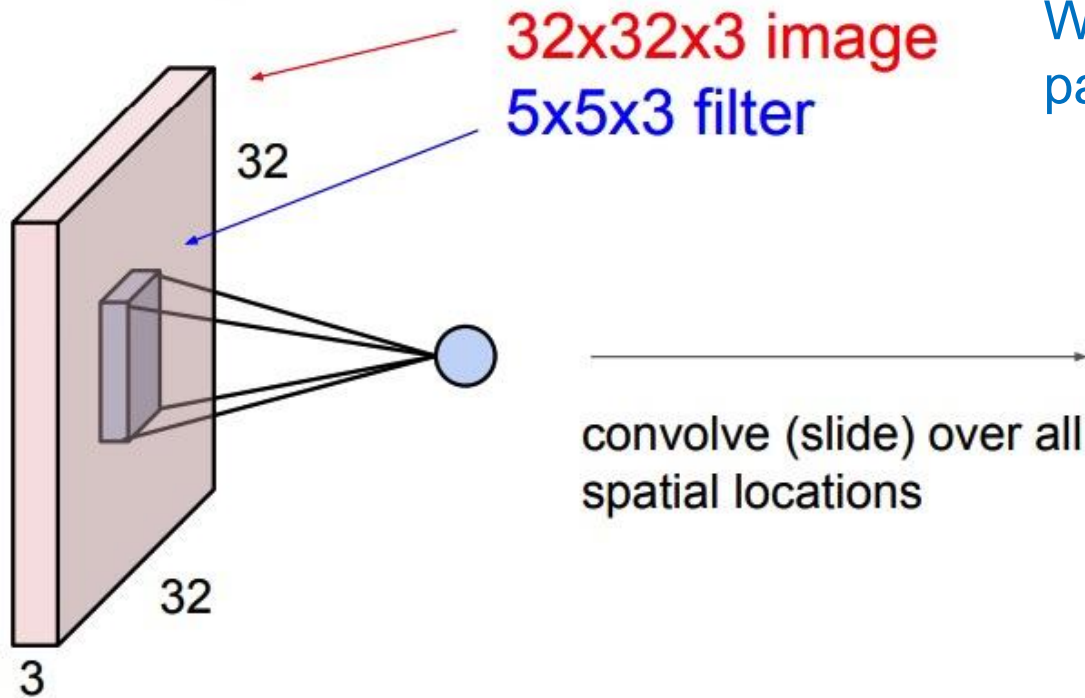
114				

Why might we want to keep the output size the same as input?

Padding: adding zeros to the input

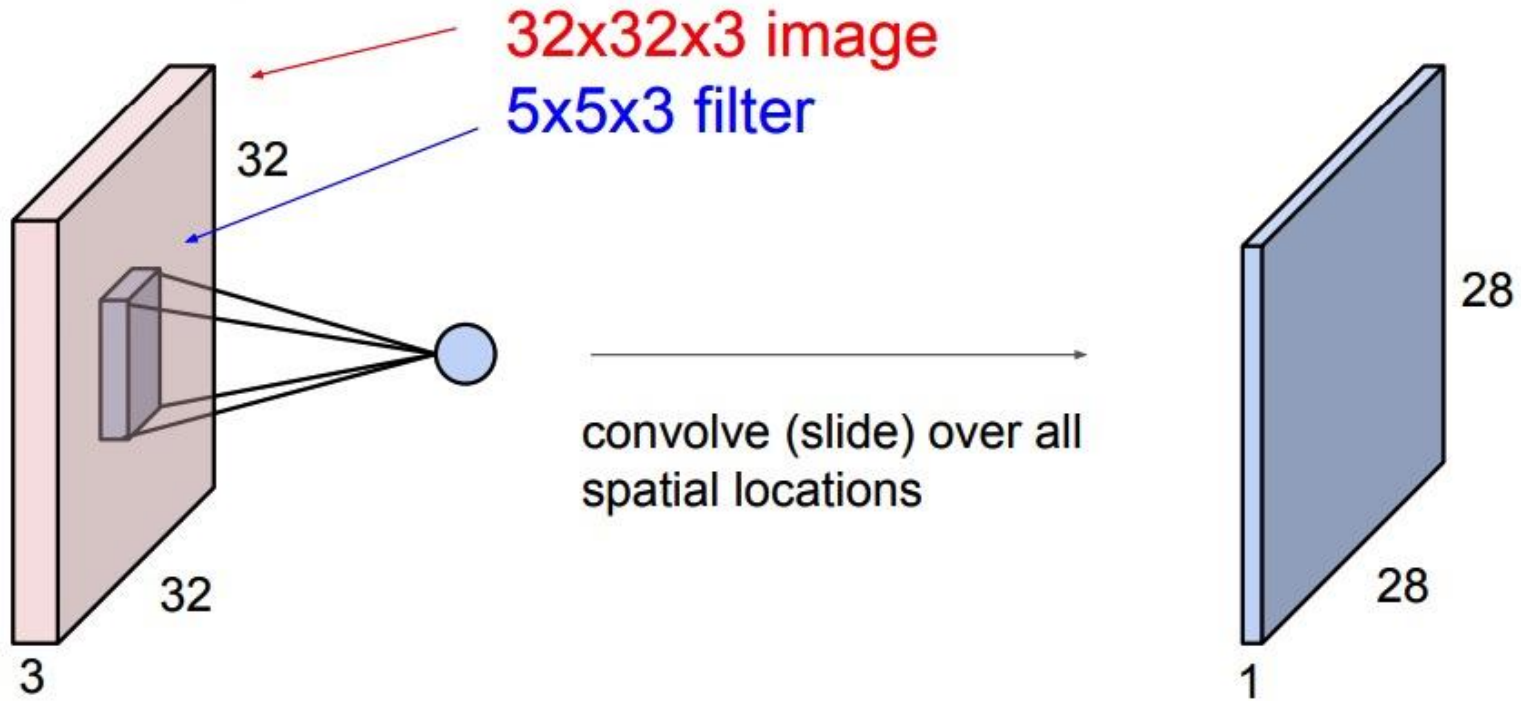
Maintain spatial resolution in deeper networks.

Convolution Layer



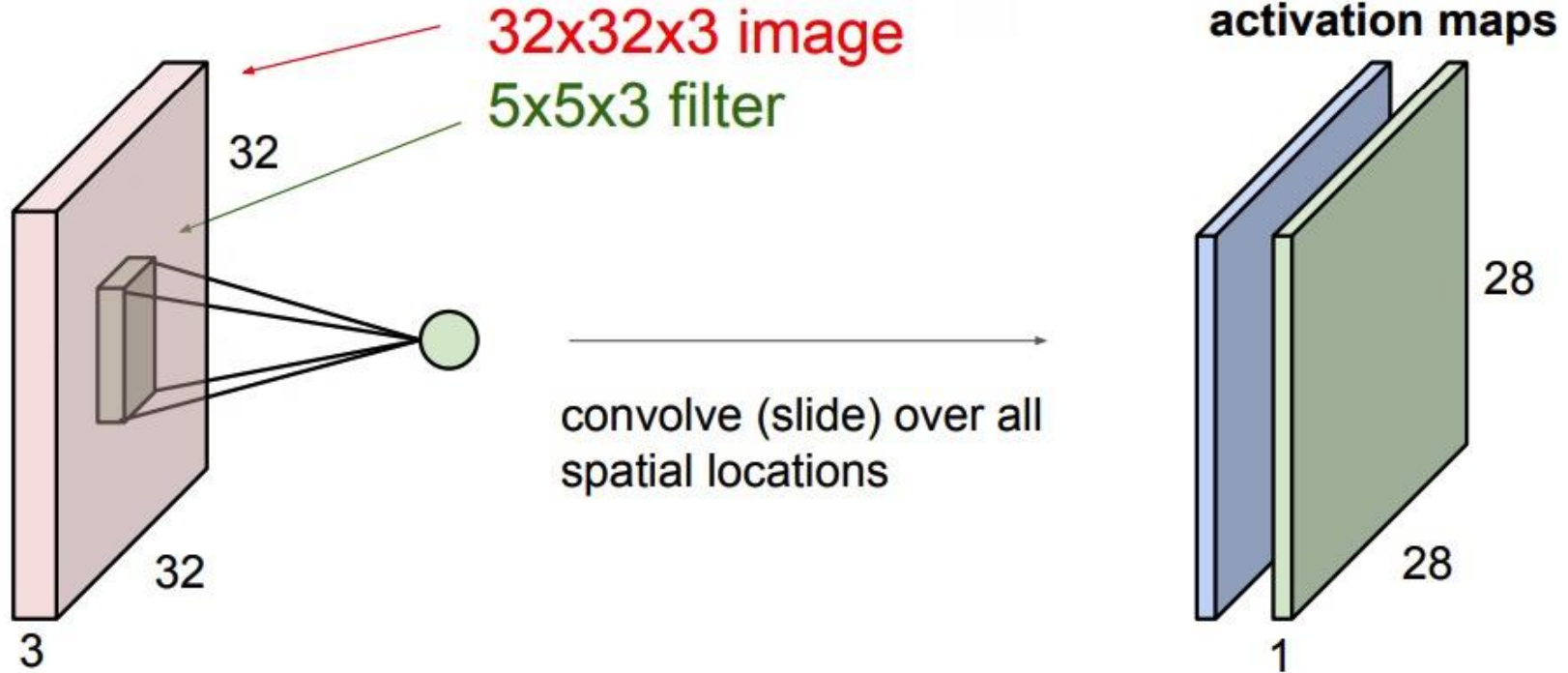
What's the output size (no padding, stride = 1)?

Convolution Layer



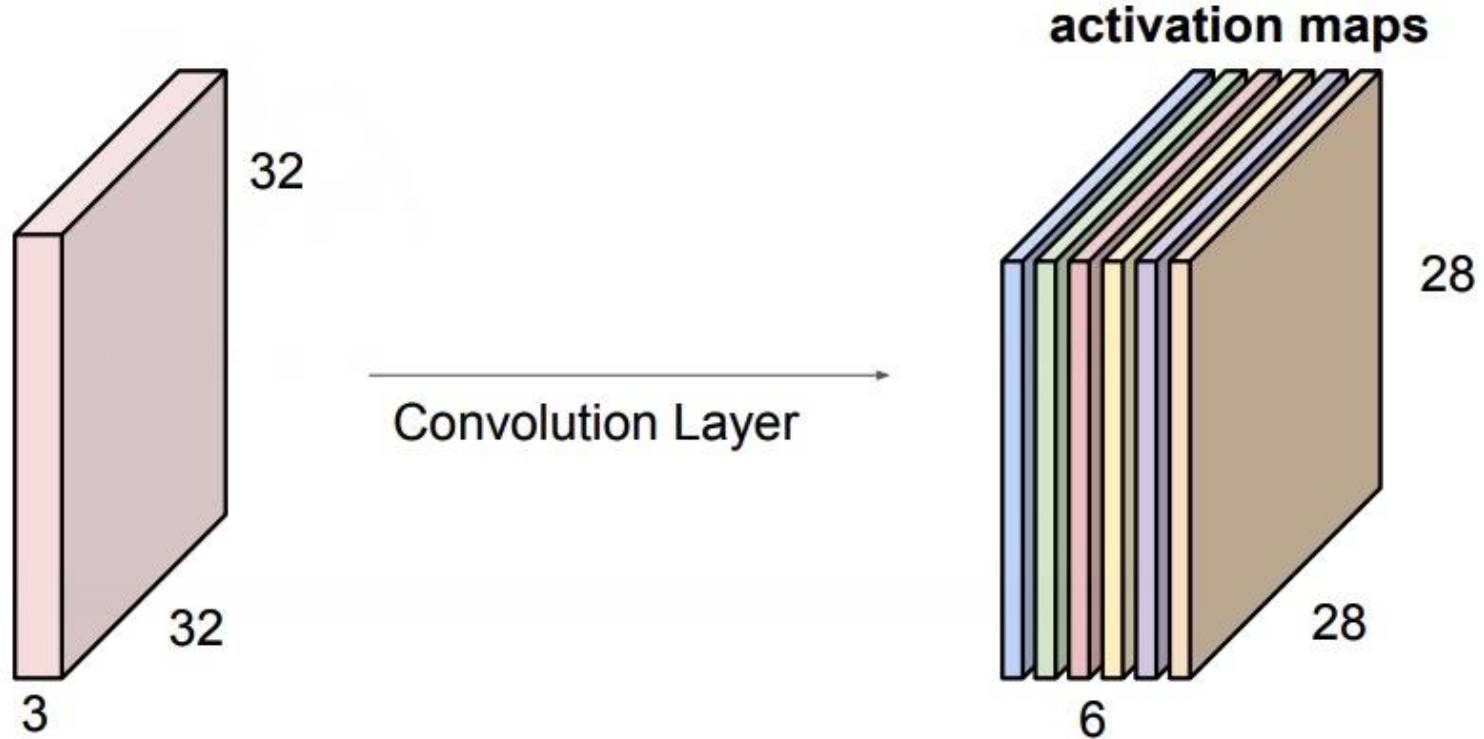
Convolution Layer

consider a second, **green** filter



Convolution Layer

For example, if we had **6 5x5x3** filters, we'll get 6 separate activation maps:



We stack these up to get a “new image” of size **28x28x6**!

Example

Input (5 × 5):

$$x = \begin{bmatrix} 1 & 1 & 2 & 0 & 0 \\ 0 & 2 & 3 & 1 & 0 \\ 1 & 2 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 1 \\ 0 & 1 & 2 & 2 & 0 \end{bmatrix}$$

Filter (3 × 3):

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

No padding, stride = 1

Example - Step 1 – Compute $y(0,0)$

$$x = \begin{bmatrix} 1 & 1 & 2 & 0 & 0 \\ 0 & 2 & 3 & 1 & 0 \\ 1 & 2 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 1 \\ 0 & 1 & 2 & 2 & 0 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\begin{aligned} y(0,0) &= 1 \times 1 + 1 \times 0 + 2 \times 1 + \\ & 0 \times 0 + 2 \times 1 + 3 \times 0 + \\ & 1 \times 1 + 2 \times 0 + 1 \times (-1) \\ &= 1 + 0 + 2 + 0 + 2 + 0 + 1 + 0 - 1 \\ &= 5 \end{aligned}$$

Example - Step 1 – Compute $y(0,1)$

$$x = \begin{bmatrix} 1 & 1 & 2 & 0 & 0 \\ 0 & 2 & 3 & 1 & 0 \\ 1 & 2 & 1 & 0 & 0 \\ 2 & 1 & 0 & 1 & 1 \\ 0 & 1 & 2 & 2 & 0 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\begin{aligned} y(0,1) &= 1 \times 1 + 2 \times 0 + 0 \times 1 + \\ & 2 \times 0 + 3 \times 1 + 1 \times 0 + \\ & 2 \times 1 + 1 \times 0 + 0 \times (-1) \\ &= 1 + 0 + 0 + 0 + 3 + 0 + 2 + 0 + 0 \\ &= 6 \end{aligned}$$

Example - Apply ReLU

$$y = \begin{bmatrix} 5 & 6 & 2 \\ 3 & 1 & -2 \\ 2 & 4 & -1 \end{bmatrix}$$

$$\text{ReLU}(y) = \max(0, y) = \begin{bmatrix} 5 & 6 & 2 \\ 3 & 1 & 0 \\ 2 & 4 & 0 \end{bmatrix}$$

Example - Padding = 1, Stride = 2

After zero-padding (1 pixel border), Input now 7×7

$$x_p = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 2 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

Compute by yourself

Example - Padding = 1, Stride = 2

$$x_p = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 2 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

$$y(0,0) = 0 \times 1 + 0 \times 0 + 0 \times 1 + 0 \times 0 + 1 \times 1 + 1 \times 0 + 0 \times 1 + 0 \times 0 + 2 \times (-1) = 0 + 0 + 0 + 0 + 1 + 0 + 0 + 0 - 2 = -1$$

$$y(0,1) = 0 \times 1 + 0 \times 0 + 0 \times 1 + 1 \times 0 + 2 \times 1 + 0 \times 0 + 2 \times 1 + 1 \times 0 + 1 \times (-1) = 0 + 0 + 0 + 0 + 2 + 0 + 2 + 0 - 1 = 3$$

$$y(0,2) = 0 \cdot 1 + 0 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0 + 0 \cdot (-1) = 1$$

Example - Padding = 1, Stride = 2

$$x_p = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 2 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$y(1,0)=2$$

$$y(1,1)=4$$

$$y(1,2)=2$$

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

Example - Padding = 1, Stride = 2

$$x_p = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 2 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$y(2,0)=1$$

What effect does stride=2 have?

$$y(2,1)=4$$

$$y(2,2)=1$$

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

Example - Padding = 1, Stride = 2

$$x_p = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 2 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

$$y(2,0)=1$$

What effect does stride=2 have?

Reduces output resolution by skipping positions.

$$y(2,1)=4$$

What effect does padding=1 have?

$$y(2,2)=1$$

Example - Padding = 1, Stride = 2

$$x_p = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 2 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$

$$y(2,0)=1$$

What effect does stride=2 have?

Reduces output resolution by skipping positions.

$$y(2,1)=4$$

What effect does padding=1 have?

$$y(2,2)=1$$

Keep the output roughly the same spatial size as input

Example - Padding = 1, Stride = 2

After Convolution Operation:

$$y = \begin{bmatrix} -1 & 3 & 1 \\ 2 & 4 & 2 \\ 1 & 4 & 1 \end{bmatrix}$$

After ReLU:

$$\text{ReLU}(y) = \begin{bmatrix} 0 & 3 & 1 \\ 2 & 4 & 2 \\ 1 & 4 & 1 \end{bmatrix}$$